คู่มือเล่ม 2: Microservices Architecture

การออกแบบ พัฒนา และดำเนินงานบริการย่อย

> ชุดคู่มือสถาปัตยกรรมซอฟต์แวร์และการบริหารโครงการ
> เล่ม 2 จาก 7 เล่ม
> ฉบับสมบูรณ์ v1.0

คำนำ

Microservices Architecture เป็นแนวทางที่ได้รับความนิยมอย่างแพร่หลายในช่วงทศวรรษที่ผ่านมา แต่ก็เป็นแนวทางที่นำมาซึ่งความซับซ้อนใหม่ที่ทีมต้องเข้าใจก่อนตัดสินใจใช้ หลายองค์กรประสบความสำเร็จกับ microservices ขณะที่อีกหลายองค์กรพบว่า monolith ที่ออกแบบดีคือคำตอบที่เหมาะสมกว่า

คู่มือเล่มนี้จึงไม่ได้มุ่งส่งเสริมให้ทุกองค์กรเปลี่ยนไปใช้ microservices แต่มีเป้าหมายเพื่อให้ผู้อ่าน **เข้าใจหลักการ ตัดสินใจได้อย่างมีเหตุผล และดำเนินงานได้อย่างปลอดภัย** เนื้อหาครอบคลุมตั้งแต่การออกแบบบริการ ขอบเขต API การเป็นเจ้าของข้อมูล การสื่อสาร Deployment Scaling Monitoring ไปจนถึงการลดความเสี่ยงและกรณีศึกษา

คู่มือเล่มนี้ต่อยอดจากเล่ม 1 (ปฐมบทสถาปัตยกรรมซอฟต์แวร์) และเชื่อมโยงกับเล่ม 3 (DDD) ซึ่งเป็นเครื่องมือสำคัญในการกำหนดขอบเขตบริการ

ข้อมูลเอกสาร

รายการ	รายละเอียด
ชื่อเล่ม	Microservices Architecture
ชื่อรอง	การออกแบบ พัฒนา และดำเนินงานบริการย่อย
กลุ่มผู้อ่าน	Software Architect, Tech Lead, Senior Developer, DevOps, SRE, PM
ระดับ	กลางถึงสูง
โครงสร้างหลัก	เรื่อง → วัตถุประสงค์ → เป้าหมาย → ข้อดี → ข้อเสีย → ข้อควรระวัง → ตัวอย่าง → การประยุกต์ใช้ → สรุป

รายการ	รายละเอียด
เล่มที่เชื่อมโยง	เล่ม 1 (พื้นฐาน), เล่ม 3 (DDD), เล่ม 4 (Clean Architecture), เล่ม 5 (ความเสี่ยง), เล่ม 6 (POC), เล่ม 7 (การบริหารโครงการ)
จำนวนบท	16 บท + 7 ภาคผนวก

วิธีใช้เอกสาร

1. **บทที่ 1–4** ใช้สร้างความเข้าใจพื้นฐานและประเมินความเหมาะสม — อ่านก่อนตัดสินใจ
2. **บทที่ 5–8** ใช้ออกแบบ API, data ownership และการสื่อสาร — อ่านก่อนลงมือออกแบบ
3. **บทที่ 9–11** ใช้วางแผน deployment, scaling และ monitoring — อ่านก่อนขึ้น production
4. **บทที่ 12–14** ใช้ประเมินความเสี่ยงและวางแนวทางลดความเสี่ยง — อ่านก่อนเริ่มโครงการ
5. **บทที่ 15–16** ใช้กรณีศึกษาและประยุกต์ใช้ — อ่านก่อนสรุปแผน
6. **ภาคผนวก** ใช้เป็นเครื่องมือประกอบการออกแบบและ review

โครงสร้างบทมาตรฐาน

ทุกบทในเล่มนี้ใช้โครงสร้างร่วมกันดังนี้

1. เรื่อง
2. วัตถุประสงค์
3. เป้าหมาย
4. หัวข้อหลัก
5. ข้อดีของบท
6. ข้อเสีย/ข้อจำกัด
7. ข้อควรระวัง
8. ตัวอย่าง
9. การประยุกต์ใช้
10. สรุป/ผลลัพธ์
11. เชื่อมโยงกับเล่มอื่น

สารบัญ

ส่วนที่ 1: ภาพรวมและพื้นฐาน Microservices

- บทที่ 1 Microservices Architecture คืออะไร
- บทที่ 2 บริการย่อย (Services)
- บทที่ 3 ขอบเขตแต่ละบริการ (Service Boundaries)
- บทที่ 4 เปรียบเทียบ Microservices กับ Monolithic

ส่วนที่ 2: การออกแบบและสื่อสารระหว่างบริการ

- บทที่ 5 API
- บทที่ 6 Data Ownership
- บทที่ 7 การสื่อสารระหว่างบริการ
- บทที่ 8 Event-Driven Communication และ Saga Pattern

ส่วนที่ 3: การดำเนินงาน (Operations)

- บทที่ 9 Deployment
- บทที่ 10 Scaling
- บทที่ 11 Monitoring และ Observability

ส่วนที่ 4: ความเสี่ยงและการลดความเสี่ยง

- บทที่ 12 ข้อดี/ข้อเสียเมื่อเทียบกับ Monolithic
- บทที่ 13 แนวทางลดความเสี่ยง
- บทที่ 14 Distributed System Challenges

ส่วนที่ 5: การนำไปใช้และกรณีศึกษา

- บทที่ 15 ตัวอย่างการออกแบบระบบ Microservices
- บทที่ 16 การประยุกต์ใช้และสรุปเล่ม

ภาคผนวก

- ภาคผนวก A: Service Boundary Canvas
- ภาคผนวก B: API Contract Checklist
- ภาคผนวก C: Data Ownership Matrix

- ภาคผนวก D: Communication Pattern Decision Record
- ภาคผนวก E: Deployment Readiness Checklist
- ภาคผนวก F: Observability Checklist
- ภาคผนวก G: Glossary เฉพาะเล่ม 2

ส่วนที่ 1: ภาพรวมและพื้นฐาน Microservices

บทที่ 1: Microservices Architecture คืออะไร

1.1 เรื่อง

ความหมาย หลักการ และที่มาของ Microservices Architecture ในฐานะแนวทางสถาปัตยกรรม

1.2 วัตถุประสงค์

- สร้างคำนิยามร่วมของ microservices
- อธิบายหลักการสำคัญที่อยู่เบื้องหลัง
- ทำให้เห็นความแตกต่างจากสถาปัตยกรรมแบบอื่น

1.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- อธิบายความหมายของ microservices ด้วยคำของตนเอง
- ระบุหลักการสำคัญได้
- เข้าใจว่า microservices ไม่ใช่ “การแบ่งโค้ดเป็นชิ้นเล็ก ๆ” เท่านั้น

1.4 หัวข้อหลัก

1.4.1 ความหมายของ Microservices

Microservices Architecture คือแนวทางที่ระบบประกอบด้วย **บริการขนาดเล็กหลายบริการ** ที่:

- **ทำงานอิสระ** — แต่ละบริการมี process ของตนเอง
- **Deploy แยกกันได้** — ไม่ต้อง deploy ทั้งระบบ
- **สื่อสารผ่าน network** — ผ่าน API หรือ message
- **มีขอบเขตธุรกิจชัดเจน** — แต่ละบริการรับผิดชอบ business capability หนึ่ง
- **เป็นเจ้าของข้อมูลของตนเอง** — ไม่ใช้ฐานข้อมูลร่วมกัน

1.4.2 หลักการสำคัญ

1. **Single Responsibility** — บริการหนึ่งรับผิดชอบธุรกิจหนึ่ง
2. **Loose Coupling** — บริการพึ่งพากันน้อยที่สุด
3. **Independent Deployability** — deploy แยกได้
4. **Decentralized Data Management** — แต่ละบริการเป็นเจ้าของข้อมูล
5. **Design for Failure** — คาดการณ์ความล้มเหลวไว้ล่วงหน้า
6. **Automation** — ใช้ automation ในทุกขั้นตอน
7. **Observability** — ตรวจสอบสถานะได้ตลอดเวลา

1.4.3 ที่มาของแนวคิด

Microservices ไม่ใช่แนวคิดใหม่ทั้งหมด แต่เป็นการรวมแนวคิดหลายอย่าง:

- SOA (Service-Oriented Architecture)
- Domain-Driven Design (DDD)
- Continuous Delivery
- DevOps
- Twelve-Factor App

1.4.4 สิ่งที่ Microservices ไม่ใช่

- ไม่ใช่ "การแบ่งโค้ดเป็นไฟล์เล็ก ๆ"
- ไม่ใช่ "การใช้ Docker ทุกอย่าง"
- ไม่ใช่ "การใช้ Kubernetes"
- ไม่ใช่คำตอบสำหรับทุกปัญหา
- ไม่ใช่เป้าหมายในตัวเอง

1.4.5 Microservices vs SOA vs Monolith

ลักษณะ	Monolith	SOA	Microservices
หน่วยการ deploy	ทั้งระบบ	บริการขนาดใหญ่	บริการขนาดเล็ก
การสื่อสาร	in-process	ESB	API/Message
ข้อมูล	ฐานข้อมูลเดียว	มักใช้ร่วม	แยกกัน
ความซับซ้อน	ต่ำ–กลาง	สูง	สูง
ทีม	ทีมเดียว	หลายทีม	หลายทีม

1.5 ข้อดีของบท

- สร้างความเข้าใจพื้นฐานที่ถูกต้อง

- ลดความเข้าใจผิดเรื่อง microservices
- ทำให้เห็นว่า microservices เป็นเครื่องมือ ไม่ใช่เป้าหมาย

1.6 ข้อเสีย/ข้อจำกัด

- เป็นภาพรวม ยังไม่ลงรายละเอียด
- ตัวอย่างเป็นระดับแนวคิด

1.7 ข้อควรระวัง

- อย่าใช้ microservices เพราะเป็นกระแส
- อย่าคิดว่า microservices แก้ปัญหาทุกอย่าง
- อย่าละเลยความซับซ้อนที่เพิ่มขึ้น

1.8 ตัวอย่าง

บริษัทหนึ่งมีระบบ monolith ที่ทีม 20 คนทำงานร่วมกัน มีปัญหา deploy บ่อยครั้งที่กระทบทั้งระบบ ทีมจึงพิจารณาแยกเป็นบริการตาม business capability (catalog, order, payment, notification) เพื่อให้แต่ละทีม deploy ได้อิสระ — นี่คือเหตุผลที่สมเหตุสมผลสำหรับ microservices

1.9 การประยุกต์ใช้

- ใช้เป็นจุดเริ่มต้นในการทำความเข้าใจ microservices
- ใช้ประเมินว่าองค์กรพร้อมหรือไม่
- ใช้สื่อสารกับผู้มีส่วนได้ส่วนเสีย

1.10 สรุป/ผลลัพธ์

ผู้อ่านเข้าใจความหมายและหลักการของ microservices และเห็นว่าเป็นแนวทางที่ต้องใช้อย่างมีเหตุผล

1.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 2–4 (รายละเอียด)
- ไปสู่เล่ม 1 (พื้นฐานสถาปัตยกรรม)
- ไปสู่เล่ม 3 (DDD สำหรับการกำหนดขอบเขต)

บทที่ 2: บริการย่อย (Services)

2.1 เรื่อง

ลักษณะของบริการย่อยใน microservices — ขนาด ความรับผิดชอบ และคุณสมบัติ

2.2 วัตถุประสงค์

- อธิบายว่าบริการย่อยคืออะไร
- กำหนดลักษณะที่ดีของบริการ
- ทำให้เห็นว่าขนาดของบริการไม่ใช่เรื่องตายตัว

2.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- อธิบายลักษณะของบริการย่อยได้
- ประเมินคุณภาพของบริการที่ออกแบบ
- กำหนดขนาดบริการที่เหมาะสม

2.4 หัวข้อหลัก

2.4.1 บริการย่อยคืออะไร

บริการคือหน่วยที่:

- **มี business capability ชัดเจน** — เช่น "จัดการคำสั่งซื้อ"
- **มี boundary** — รู้ว่าอะไรอยู่ข้างใน อะไรอยู่ข้างนอก
- **มี API** — สื่อสารกับภายนอกผ่าน contract
- **มีข้อมูลของตนเอง** — ไม่แชร์ฐานข้อมูล
- **Deploy ได้อิสระ** — ไม่ขึ้นกับบริการอื่น

2.4.2 ขนาดของบริการ

ไม่มีกฎตายตัว แต่มีหลักการ:

- **เล็กพอที่จะทีมเล็กดูแลได้** — ทีม 2–8 คน
- **ใหญ่พอที่จะมี business value** — ไม่ใช่แค่ CRUD
- **ไม่เล็กจนเกินไป** — จนต้องสื่อสารกันตลอดเวลา
- **ไม่ใหญ่จนเกินไป** — จนกลายเป็น monolith ย่อส่วน

"ขนาดที่เหมาะสม" ขึ้นกับบริบท: ทีม, โดเมน, เทคโนโลยี

2.4.3 คุณสมบัติของบริการที่ดี

- **Autonomous** — ทำงานได้ด้วยตนเอง
- **Loosely coupled** — พึ่งพาน้อย
- **Highly cohesive** — สิ่งที่เกี่ยวข้องกันอยู่ด้วยกัน
- **Observable** — ตรวจสอบสถานะได้
- **Testable** — ทดสอบได้
- **Deployable** — deploy ได้อิสระ

2.4.4 บริการกับ Bounded Context

บริการที่ดีมักสอดคล้องกับ bounded context (ดูเล่ม 3):

- หนึ่ง bounded context อาจเป็นหนึ่งบริการ
- หรือหลายบริการถ้าซับซ้อน
- การกำหนดขอบเขตบริการควรอิงโดเมน ไม่ใช่อิงเทคโนโลยี

2.4.5 Anti-patterns ของบริการ

- **God service** — บริการที่ทำ太多
- **Nano service** — บริการที่เล็กเกินไป
- **Shared database** — หลายบริการใช้ฐานข้อมูลเดียวกัน
- **Distributed monolith** — แยกบริการแต่ coupling สูง

2.5 ข้อดีของบท

- ทำให้เห็นลักษณะของบริการที่ดี
- ลด anti-patterns
- เชื่อมโยงกับ DDD

2.6 ข้อเสีย/ข้อจำกัด

- ขนาดบริการขึ้นกับบริบท
- ตัวอย่างอาจไม่ครอบคลุมทุกกรณี

2.7 ข้อควรระวัง

- อย่าแบ่งบริการตามเทคโนโลยี
- อย่าแบ่งเล็กเกินไปจนกลายเป็น distributed monolith
- อย่าละเลย business capability

2.8 ตัวอย่าง

ระบบ e-commerce แบ่งเป็นบริการ:

- **Catalog Service** — จัดการสินค้า
- **Order Service** — จัดการคำสั่งซื้อ
- **Payment Service** — จัดการชำระเงิน
- **Notification Service** — แจ้งเตือน

แต่ละบริการมี business capability ชัดเจน มีข้อมูลของตนเอง และ deploy แยกได้

2.9 การประยุกต์ใช้

- ใช้กำหนดบริการในระบบใหม่

- ใช้ทบทวนบริการที่มีอยู่
- ใช้ใน workshop ออกแบบบริการ

2.10 สรุป/ผลลัพธ์

ผู้อ่านอธิบายลักษณะของบริการย่อยได้ และประเมินคุณภาพของบริการที่ออกแบบ

2.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 3 (ขอบเขตบริการ)
- ไปสู่เล่ม 3 (DDD bounded context)

บทที่ 3: ขอบเขตแต่ละบริการ (Service Boundaries)

3.1 เรื่อง

การกำหนดขอบเขตของบริการ — หลักการ วิธีการ และเครื่องมือ

3.2 วัตถุประสงค์

- อธิบายหลักการกำหนดขอบเขตบริการ
- ทำให้เห็นความสำคัญของ boundary ที่ดี
- เชื่อมโยงกับ DDD และ business capability

3.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- กำหนดขอบเขตบริการได้อย่างมีเหตุผล
- ใช้เครื่องมือช่วยกำหนดขอบเขต
- หลีกเลี่ยง boundary ที่ผิดพลาด

3.4 หัวข้อหลัก

3.4.1 หลักการกำหนดขอบเขต

- **Business Capability** — บริการควรสอดคล้องกับความสามารถทางธุรกิจ
- **Bounded Context** — ขอบเขตของโมเดลและภาษา
- **Data Ownership** — ใครเป็นเจ้าของข้อมูลอะไร
- **Team Structure** — ทีมใดรับผิดชอบบริการใด (Conway's Law)
- **Change Frequency** — สิ่งที่เปลี่ยนบ่อยด้วยกันควรอยู่ด้วยกัน

3.4.2 วิธีการกำหนดขอบเขต

1. **Event Storming** — ระดมสมองหา domain events
2. **Domain Storytelling** — เล่าเรื่อง业务流程
3. **Context Mapping** — map ความสัมพันธ์ระหว่าง bounded context
4. **Business Capability Mapping** — แยกความสามารถทางธุรกิจ
5. **CRC Cards** — Class-Responsibility-Collaboration

3.4.3 ขอบเขตที่ดี vs ไม่ดี

ขอบเขตที่ดี:

- มี business capability ชัดเจน
- มีข้อมูลของตนเอง
- มี API ที่ stable
- มีทีมรับผิดชอบชัดเจน
- เปลี่ยนได้โดยไม่กระทบบริการอื่นมาก

ขอบเขตที่ไม่ดี:

- แบ่งตามเทคโนโลยี (UI service, DB service)
- แบ่งเล็กเกินไปจนต้องสื่อสารกันตลอด
- มี shared database
- มี cyclic dependency ระหว่างบริการ

3.4.4 Conway's Law

"องค์กรจะออกแบบระบบให้สะท้อนโครงสร้างการสื่อสารขององค์กร"

นัยสำคัญ:

- โครงสร้างทีมส่งผลต่อสถาปัตยกรรม
- ถ้าต้องการสถาปัตยกรรมแบบใด ต้องจัดทีมให้สอดคล้อง
- ทีมที่ดูแลบริการควรมีอำนาจตัดสินใจเต็มที่ (full ownership)

3.4.5 การทบทวนขอบเขต

ขอบเขตบริการไม่ใช่เรื่องถาวร ต้องทบทวนเมื่อ:

- ธุรกิจเปลี่ยน
- ทีมเปลี่ยน
- ปัญหา coupling เกิดขึ้น
- มีข้อมูลใหม่จาก production

3.5 ข้อดีของบท

- ทำให้เห็นความสำคัญของ boundary

- ให้เครื่องมือและวิธีการ
- เชื่อมโยงกับ DDD และโครงสร้างทีม

3.6 ข้อเสีย/ข้อจำกัด

- ต้องอาศัยความเข้าใจโดเมน
- ต้องใช้เวลาและผู้มีส่วนได้ส่วนเสีย

3.7 ข้อควรระวัง

- อย่าแบ่งตามเทคโนโลยี
- อย่าละเลย Conway's Law
- อย่าคิดว่าขอบเขตเป็นเรื่องถาวร

3.8 ตัวอย่าง

ระบบ e-commerce:

- **Order Service** — รับผิดชอบ lifecycle ของคำสั่งซื้อ
- **Payment Service** — รับผิดชอบการชำระเงิน
- **Inventory Service** — รับผิดชอบสต็อก

Order Service ไม่ควรเข้าถึงฐานข้อมูลของ Payment Service โดยตรง แต่ควรเรียกผ่าน API

3.9 การประยุกต์ใช้

- ใช้ workshop Event Storming
- ใช้ Context Mapping
- ใช้ทบทวนขอบเขตบริการเป็นระยะ

3.10 สรุป/ผลลัพธ์

ผู้อ่านกำหนดขอบเขตบริการได้อย่างมีเหตุผล และใช้เครื่องมือช่วยได้

3.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 4–6
- ไปสู่เล่ม 3 (DDD, Context Mapping)

บทที่ 4: เปรียบเทียบ Microservices กับ Monolithic

4.1 เรื่อง

การเปรียบเทียบข้อดี/ข้อเสียของ microservices กับ monolith เพื่อการตัดสินใจ

4.2 วัตถุประสงค์

- ทำให้เห็น trade-offs อย่างชัดเจน
- ช่วยตัดสินใจได้อย่างมีเหตุผล
- ลดการเลือกตามกระแส

4.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- เปรียบเทียบสองแนวทางได้
- ตัดสินใจได้อย่างมีเหตุผล
- รู้ว่าเมื่อใดควรเลือกแบบใด

4.4 หัวข้อหลัก

4.4.1 ตารางเปรียบเทียบ

มิติ	Monolith	Microservices
ความซับซ้อนเริ่มต้น	ต่ำ	สูง
ความซับซ้อนระยะยาว	สูงเมื่อโต	สูงตั้งแต่ต้น
Deploy	ทั้งระบบ	แยกบริการ
Scalability	ทั้งก้อน	แต่ละบริการ
Technology	stack เดียว	หลาย stack
Testing	integration ง่าย	ต้องมี contract test
Debugging	stack trace ต่อเนื่อง	distributed tracing
Data consistency	ACID	eventual consistency
ทีม	ทีมเดียว	หลายทีม
Time-to-market (เริ่มต้น)	เร็ว	ช้า

มิติ	Monolith	Microservices
Time-to-market (ระยะยาว)	อาจช้าลง	เร็วขึ้นถ้าทำดี
ค่าใช้จ่าย infrastructure	ต่ำ	สูง
ต้องการ DevOps maturity	ต่ำ	สูง

4.4.2 เมื่อใดควรเลือก Monolith

- ระบบขนาดเล็ก–กลาง
- ทีมเล็ก (น้อยกว่า 10 คน)
- ช่วงเริ่มต้นที่ requirements ไม่ชัด
- ระบบที่ไม่ต้องการ scaling สูง
- องค์กรที่ยังไม่มี DevOps maturity
- ระบบที่มี business logic ซับซ้อนแต่ไม่ต้องการ distributed complexity

4.4.3 เมื่อใดควรเลือก Microservices

- ระบบใหญ่ที่มีหลาย business capability
- ทีมหลายทีมที่ต้องการ deploy อิสระ
- ต้องการ scaling เฉพาะบางส่วน
- ต้องการ technology diversity
- องค์กรมี DevOps maturity
- มี bounded context ชัดเจน

4.4.4 ทางเลือกกลาง: Modular Monolith

- เริ่มด้วย monolith ที่แบ่ง module ชัดเจน
- เตรียมพร้อมสำหรับการแยกในอนาคต
- ลดความเสี่ยงของการแยกเร็วเกินไป
- เหมาะกับหลายองค์กร

4.4.5 การตัดสินใจ

การตัดสินใจควรอิง:

- Architectural drivers (ดูเล่ม 1 บทที่ 5)
- ความพร้อมของทีมและองค์กร
- งบประมาณและเวลา
- ความเสี่ยงที่ยอมรับได้

4.5 ข้อดีของบท

- ทำให้เห็น trade-offs อย่างชัดเจน

- ช่วยตัดสินใจได้อย่างมีเหตุผล
- ลดการเลือกตามกระแส

4.6 ข้อเสีย/ข้อจำกัด

- การเปรียบเทียบเป็นเรื่อง contextual
- ตัวอย่างอาจไม่ครอบคลุมทุกกรณี

4.7 ข้อควรระวัง

- อย่าเลือก microservices เพราะเป็นกระแส
- อย่าคิดว่า monolith เป็นสิ่งเลวร้าย
- อย่าละเลยความพร้อมขององค์กร

4.8 ตัวอย่าง

กรณีที่ 1: Startup ระยะเริ่มต้น ทีม 5 คน ระบบ e-commerce เล็ก ๆ → **เลือก modular monolith**

กรณีที่ 2: องค์กรใหญ่ มีหลายทีม แต่ละทีมดูแล business capability ต่างกัน ต้องการ deploy อิสระ → **เลือก microservices**

กรณีที่ 3: ระบบ ERP ขนาดกลาง ทีม 15 คน มี business logic ซับซ้อน → **เลือก modular monolith ก่อน แล้วพิจารณาแยกเฉพาะส่วนที่ต้องการ**

4.9 การประยุกต์ใช้

- ใช้ใน workshop ตัดสินใจสถาปัตยกรรม
- ใช้เป็น checklist ก่อนเริ่มโครงการ
- ใช้สื่อสารกับผู้มีส่วนได้ส่วนเสีย

4.10 สรุป/ผลลัพธ์

ผู้อ่านเปรียบเทียบสองแนวทางได้ และตัดสินใจได้อย่างมีเหตุผล

4.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 5–14 (รายละเอียด microservices)
- ไปสู่เล่ม 1 (Monolithic Architecture)
- ไปสู่เล่ม 5 (ความเสี่ยง)

ส่วนที่ 2: การออกแบบและสื่อสารระหว่างบริการ

บทที่ 5: API

5.1 เรื่อง

การออกแบบ API สำหรับ microservices — contract, versioning, documentation และ security

5.2 วัตถุประสงค์

- อธิบายบทบาทของ API ใน microservices
- สอนการออกแบบ API ที่ดี
- ทำให้เห็นความสำคัญของ contract

5.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ออกแบบ API ที่ stable และสื่อความหมาย
- กำหนด versioning strategy
- จัดทำ documentation และ security

5.4 หัวข้อหลัก

5.4.1 API คืออะไรในบริบท Microservices

API คือ **contract** ระหว่างบริการ:

- กำหนดว่าใครเรียกอะไรได้
- กำหนดรูปแบบ request/response
- เป็นสัญญาที่ต้องรักษา
- เปลี่ยนแปลงได้แต่ต้องมีกลไก

5.4.2 ประเภทของ API

- **REST** — resource-based, HTTP verbs
- **gRPC** — binary, performance สูง
- **GraphQL** — query language, flexible
- **Message-based** — async ผ่าน queue/topic

การเลือกขึ้นกับ:

- ความต้องการ performance
- ความต้องการ flexibility
- ทักษะทีม
- ecosystem

5.4.3 การออกแบบ API ที่ดี

- **Resource-oriented** — ใช้ nouns ไม่ใช่ verbs
- **Consistent** — รูปแบบเดียวกันทั้งระบบ
- **Stateless** — ไม่เก็บ state ระหว่าง request
- **Versioned** — มี strategy ชัดเจน
- **Documented** — มี OpenAPI/Swagger
- **Secured** — มี authentication/authorization
- **Idempotent** — สำหรับ operation ที่ควรเป็นเช่นนั้น

5.4.4 Versioning Strategy

- **URI versioning** — `/v1/orders`
- **Header versioning** — `Accept: application/vnd.api.v1+json`
- **Query parameter** — `?version=1`

หลักการ:

- หลีกเลี่ยง breaking changes
- ถ้าจำเป็นต้องมี ให้เพิ่ม version ใหม่
- Maintain version เก่าในระยะเปลี่ยนผ่าน
- ประกาศ deprecation ล่วงหน้า

5.4.5 Documentation

- ใช้ OpenAPI/Swagger
- มี examples
- อัปเดตอัตโนมัติจาก code
- เข้าถึงได้จาก portal

5.4.6 Security

- Authentication (OAuth2, JWT)
- Authorization (RBAC, ABAC)
- Rate limiting
- Input validation
- HTTPS เท่านั้น

5.5 ข้อดีของบท

- ทำให้เห็นความสำคัญของ API contract
- ให้แนวทางปฏิบัติที่ชัดเจน
- ลดปัญหา breaking changes

5.6 ข้อเสีย/ข้อจำกัด

- ตัวอย่างอาจไม่ครอบคลุมทุกภาษา
- ต้องปรับตาม ecosystem

5.7 ข้อควรระวัง

- อย่าปล่อยให้ API เปลี่ยนโดยไม่แจ้ง
- อย่าละเลย versioning
- อย่าละเลย security
- อย่าให้ API ผูกกับ implementation ภายใน

5.8 ตัวอย่าง

API ของ Order Service:

```
GET /v1/orders/{id}
POST /v1/orders
PUT /v1/orders/{id}/status
```

Response:

```
{
  "id": "ord-123",
  "customerId": "cust-456",
  "items": [...],
  "status": "CONFIRMED",
  "createdAt": "2026-10-30T10:00:00Z"
}
```

5.9 การประยุกต์ใช้

- ใช้กำหนด API contract ก่อนพัฒนา
- ใช้ใน API review
- ใช้เป็น contract test

5.10 สรุป/ผลลัพธ์

ผู้อ่านออกแบบ API ที่ stable สื่อความหมาย และมี versioning ได้

5.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 6–7
- ไปสู่เล่ม 1 (Contract ใน component-based)

บทที่ 6: Data Ownership

6.1 เรื่อง

การเป็นเจ้าของข้อมูลใน microservices — หลักการ รูปแบบ และการจัดการ

6.2 วัตถุประสงค์

- อธิบายหลักการ data ownership
- ทำให้เห็นความสำคัญของการแยกข้อมูล
- สอนการจัดการข้อมูลใน distributed system

6.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- กำหนดเจ้าของข้อมูลได้
- ออกแบบ data ownership ได้
- จัดการ consistency ได้อย่างเหมาะสม

6.4 หัวข้อหลัก

6.4.1 หลักการ Data Ownership

แต่ละบริการเป็นเจ้าของข้อมูลของตนเอง หมายความว่า:

- บริการเดียวเท่านั้นที่เขียนข้อมูลนั้นได้
- บริการอื่นเข้าถึงผ่าน API
- ไม่มี shared database
- ไม่มีการ query ข้ามฐานข้อมูลโดยตรง

6.4.2 ทำไมต้องแยกข้อมูล

- **Loose coupling** — บริการไม่ผูกกันด้วย schema
- **Independent evolution** — แต่ละบริการเปลี่ยน schema ได้
- **Clear ownership** — ใครรับผิดชอบอะไรชัดเจน
- **Scalability** — แต่ละบริการ scale ฐานข้อมูลของตนเอง

6.4.3 รูปแบบ Data Ownership

- **Database per Service** — แต่ละบริการมีฐานข้อมูลของตนเอง
- **Schema per Service** — ใช้ฐานข้อมูลร่วมแต่แยก schema (ทางเลือกกลาง)
- **Shared Database** — หลายบริการใช้ฐานข้อมูลเดียว (anti-pattern)

6.4.4 ความท้าทาย

- **Distributed transactions** — ไม่มี ACID ข้ามบริการ
- **Data consistency** — ต้องใช้ eventual consistency
- **Reporting** — ต้องรวมข้อมูลจากหลายบริการ
- **Data duplication** — อาจต้องเก็บข้อมูลซ้ำ

6.4.5 แนวทางจัดการ

- **Saga Pattern** — จัดการ distributed transaction (ดูบทที่ 8)
- **Event Sourcing** — เก็บ events เป็น source of truth
- **CQRS** — แยก read/write model
- **Data Lake / Warehouse** — สำหรับ reporting
- **API Composition** — รวมข้อมูลผ่าน API

6.5 ข้อดีของบท

- ทำให้เห็นความสำคัญของ data ownership
- ให้แนวทางจัดการความท้าทาย
- ลดปัญหา coupling ผ่านข้อมูล

6.6 ข้อเสีย/ข้อจำกัด

- ต้องใช้ความพยายามในการออกแบบ
- มีความซับซ้อนเพิ่มขึ้น

6.7 ข้อควรระวัง

- อย่าใช้ shared database
- อย่าละเลย eventual consistency
- อย่าละเลย reporting requirements
- อย่า duplicate ข้อมูลโดยไม่มีเหตุผล

6.8 ตัวอย่าง

ระบบ e-commerce:

- **Order Service** — เป็นเจ้าของข้อมูลคำสั่งซื้อ
- **Payment Service** — เป็นเจ้าของข้อมูลการชำระเงิน
- **Inventory Service** — เป็นเจ้าของข้อมูลสต็อก

Order Service ไม่ควร query ฐานข้อมูล Payment โดยตรง แต่เรียกผ่าน API หรือรับ event

6.9 การประยุกต์ใช้

- ใช้กำหนด data ownership matrix (ภาคผนวก C)
- ใช้วางแผน migration จาก shared database
- ใช้ใน architecture review

6.10 สรุป/ผลลัพธ์

ผู้อ่านกำหนดเจ้าของข้อมูลได้ และจัดการ consistency ได้อย่างเหมาะสม

6.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 7–8
- ไปสู่เล่ม 3 (Aggregate และ Repository)

บทที่ 7: การสื่อสารระหว่างบริการ

7.1 เรื่อง

รูปแบบการสื่อสารระหว่างบริการ — synchronous, asynchronous และการเลือกใช้

7.2 วัตถุประสงค์

- อธิบายรูปแบบการสื่อสาร
- ทำให้เห็น trade-offs
- สอนการเลือกรูปแบบที่เหมาะสม

7.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- อธิบายรูปแบบการสื่อสารได้
- เลือกรูปแบบที่เหมาะสมกับสถานการณ์
- ออกแบบการสื่อสารที่ resilient

7.4 หัวข้อหลัก

7.4.1 Synchronous Communication

- ผู้เรียกส่ง request และรอ response
- ตัวอย่าง: REST, gRPC
- **ข้อดี:** ง่าย, เข้าใจง่าย, result ทันที
- **ข้อเสีย:** coupling สูง, ถ้า downstream ล่ม ผู้เรียกก็ล่ม

7.4.2 Asynchronous Communication

- ผู้ส่งส่ง message และไม่รอ
- ตัวอย่าง: message queue (RabbitMQ, Kafka)
- **ข้อดี:** decoupling, resilient, scalable
- **ข้อเสีย:** ซับซ้อน, debug ยาก, eventual consistency

7.4.3 รูปแบบการสื่อสาร

- **Request/Response** — synchronous
- **Event Notification** — async, แจ้งว่าเกิดอะไร
- **Event-Carried State Transfer** — async, ส่งข้อมูลใน event
- **Command** — async, สั่งให้ทำอะไร
- **Publish/Subscribe** — หลายผู้รับ
- **Point-to-Point** — ผู้รับเดียว

7.4.4 การเลือกรูปแบบ

สถานการณ์	แนะนำ
ต้องการผลทันที	Synchronous
ต้องการ decoupling	Asynchronous
หลายผู้รับ	Publish/Subscribe
ต้องการ resilient	Asynchronous + retry
ต้องการ simplicity	Synchronous

7.4.5 Resilience Patterns

- **Timeout** — จำกัดเวลารอ
- **Retry** — ลองใหม่เมื่อล้มเหลว
- **Circuit Breaker** — หยุดเรียกเมื่อ downstream ล่ม
- **Bulkhead** — แยก resource pool
- **Fallback** — ใช้ทางเลือกเมื่อล้มเหลว

7.5 ข้อดีของบท

- ทำให้เห็น trade-offs อย่างชัดเจน
- ให้แนวทางเลือกที่เหมาะสม
- เตรียมพื้นฐานสำหรับ resilience

7.6 ข้อเสีย/ข้อจำกัด

- ตัวอย่างอาจไม่ครอบคลุมทุกเทคโนโลยี
- ต้องปรับตาม ecosystem

7.7 ข้อควรระวัง

- อย่าใช้ synchronous ทุกที่
- อย่าใช้ asynchronous โดยไม่เข้าใจ eventual consistency
- อย่าละเลย resilience patterns
- อย่าละเลย idempotency

7.8 ตัวอย่าง

Order Service → Payment Service:

- **Synchronous:** เรียก REST API เพื่อชำระเงิน
- **Asynchronous:** ส่ง event `OrderCreated` ให้ Payment Service ประมวลผล

การเลือกขึ้นกับ business requirement และ resilience requirement

7.9 การประยุกต์ใช้

- ใช้กำหนด communication pattern (ภาคผนวก D)
- ใช้ใน architecture review
- ใช้ใน POC เพื่อทดสอบ

7.10 สรุป/ผลลัพธ์

ผู้อ่านอธิบายรูปแบบการสื่อสารได้ และเลือกรูปแบบที่เหมาะสม

7.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 8
- ไปสู่เล่ม 5 (ความเสี่ยงจาก distributed system)

บทที่ 8: Event-Driven Communication และ Saga Pattern

8.1 เรื่อง

การสื่อสารแบบ event-driven และการจัดการ distributed transaction ด้วย Saga

8.2 วัตถุประสงค์

- อธิบาย event-driven architecture
- สอน Saga pattern
- ทำให้เห็นวิธีจัดการ consistency ใน distributed system

8.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ออกแบบ event-driven communication ได้
- ใช้ Saga pattern จัดการ transaction
- เข้าใจ eventual consistency

8.4 หัวข้อหลัก

8.4.1 Event-Driven Architecture

- องค์ประกอบสื่อสารผ่าน events
- Event คือ "สิ่งที่เกิดขึ้นแล้ว" (past tense)
- ผู้ส่งไม่รู้จักผู้รับ
- **ข้อดี:** decoupling, scalable, resilient
- **ข้อเสีย:** ซับซ้อน, debug ยาก

8.4.2 ประเภทของ Events

- **Domain Event** — เหตุการณ์ทางธุรกิจ เช่น `OrderPlaced`
- **Integration Event** — event สำหรับสื่อสารข้ามบริการ
- **System Event** — เหตุการณ์ทางเทคนิค

8.4.3 Event Sourcing

- เก็บ events เป็น source of truth
- State ปัจจุบันสร้างจาก events
- **ข้อดี:** audit trail, replay ได้
- **ข้อเสีย:** ซับซ้อน, query ยาก

8.4.4 Saga Pattern

Saga คือลำดับของ local transactions ที่ประสานกัน:

- **Choreography** — แต่ละบริการฟัง event และตัดสินใจเอง
- **Orchestration** — มี orchestrator สั่งการ

Choreography:

- ไม่มีศูนย์กลาง

- ยืดหยุ่น แต่ติดตามยาก
- เหมาะกับ saga ที่ไม่ซับซ้อน

Orchestration:

- มี orchestrator
- ติดตามง่าย แต่มี single point
- เหมาะกับ saga ที่ซับซ้อน

8.4.5 compensating Transactions

เมื่อ saga ล้มเหลว ต้องมี compensating action:

- `OrderCreated` → `OrderCancelled`
- `PaymentProcessed` → `PaymentRefunded`
- `InventoryReserved` → `InventoryReleased`

8.4.6 Eventual Consistency

- ข้อมูลจะสอดคล้องกันในที่สุด
- ไม่ทันทีเหมือน ACID
- ต้องออกแบบ UI/UX ให้รองรับ
- ต้องมี monitoring

8.5 ข้อดีของบท

- ทำให้เห็นวิธีจัดการ distributed transaction
- ให้แนวทางที่ใช้ได้จริง
- เตรียมพื้นฐานสำหรับระบบที่ซับซ้อน

8.6 ข้อเสีย/ข้อจำกัด

- Saga มีความซับซ้อน
- ต้องออกแบบ compensating actions อย่างระมัดระวัง

8.7 ข้อควรระวัง

- อย่าละเลย compensating actions
- อย่าละเลย idempotency
- อย่าละเลย monitoring
- อย่าคิดว่า eventual consistency ง่าย

8.8 ตัวอย่าง

Saga สำหรับการสั่งซื้อ:

1. Order Service สร้าง order (`OrderCreated`)
2. Payment Service ประมวลผล (`PaymentProcessed`)
3. Inventory Service จองสต็อก (`InventoryReserved`)
4. Order Service ยืนยัน (`OrderConfirmed`)

ถ้า Inventory ล้มเหลว:

- Payment Service ต้อง refund (`PaymentRefunded`)
- Order Service ต้องยกเลิก (`OrderCancelled`)

8.9 การประยุกต์ใช้

- ใช้ในระบบที่มี distributed transaction
- ใช้ใน POC เพื่อทดสอบ
- ใช้ใน architecture review

8.10 สรุป/ผลลัพธ์

ผู้อ่านออกแบบ event-driven communication และใช้ Saga pattern ได้

8.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 9–11
- ไปสู่เล่ม 4 (CQRS และ Event Sourcing)

ส่วนที่ 3: การดำเนินงาน (Operations)

บทที่ 9: Deployment

9.1 เรื่อง

การ deploy microservices — containerization, orchestration และ CI/CD

9.2 วัตถุประสงค์

- อธิบายแนวทาง deploy microservices
- ทำให้เห็นความสำคัญของ automation
- เตรียมพื้นฐานสำหรับ production

9.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- วางแผน deployment ได้
- เลือกเทคโนโลยีที่เหมาะสม
- ออกแบบ CI/CD pipeline

9.4 หัวข้อหลัก

9.4.1 Containerization

- Docker เป็นมาตรฐาน
- แต่ละบริการมี container image ของตนเอง
- Image ควรเล็ก, immutable, versioned
- ใช้ multi-stage build

9.4.2 Orchestration

- Kubernetes เป็นมาตรฐาน
- จัดการ scaling, rolling update, service discovery
- ทางเลือก: Docker Swarm, Nomad, ECS

9.4.3 CI/CD Pipeline

- **CI:** build, test, static analysis
- **CD:** deploy to staging, smoke test, deploy to production
- **Blue/Green:** deploy version ใหม่เคียง version เก่า
- **Canary:** deploy ให้บางส่วนก่อน
- **Rolling:** deploy ทีละส่วน

9.4.4 Configuration Management

- ใช้ environment variables
- ใช้ config server (Spring Cloud Config, Consul)
- อย่า bake config ลง image
- ใช้ secrets management (Vault, Kubernetes Secrets)

9.4.5 Deployment Readiness Checklist

- มี health check
- มี readiness/liveness probe
- มี graceful shutdown
- มี resource limits
- มี logging และ monitoring

- มี rollback plan

9.5 ข้อดีของบท

- ทำให้เห็นภาพรวม deployment
- ให้ checklist ที่ใช้ได้จริง
- เตรียมพื้นฐานสำหรับ production

9.6 ข้อเสีย/ข้อจำกัด

- ตัวอย่างอาจไม่ครอบคลุมทุกเทคโนโลยี
- ต้องปรับตาม ecosystem

9.7 ข้อควรระวัง

- อย่า deploy โดยไม่มี health check
- อย่า bake config ลง image
- อย่าละเลย rollback plan
- อย่าละเลย secrets management

9.8 ตัวอย่าง

CI/CD Pipeline สำหรับ Order Service:

1. **CI:** lint, unit test, build image
2. **Push:** push image to registry
3. **Deploy Staging:** deploy to staging
4. **Smoke Test:** ทดสอบ basic flows
5. **Deploy Production:** canary 10% → 50% → 100%
6. **Monitor:** ตรวจสอบ metrics

9.9 การประยุกต์ใช้

- ใช้ในโครงการใหม่
- ใช้ปรับปรุง pipeline เดิม
- ใช้ใน architecture review

9.10 สรุป/ผลลัพธ์

ผู้อ่านวางแผน deployment ได้ และออกแบบ CI/CD pipeline

9.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 10–11

- ไปสู่เล่ม 7 (การบริหารโครงการ)

บทที่ 10: Scaling

10.1 เรื่อง

การ scaling microservices — horizontal, vertical และ auto-scaling

10.2 วัตถุประสงค์

- อธิบายแนวทาง scaling
- ทำให้เห็นความสำคัญของ stateless
- เตรียมพื้นฐานสำหรับ production ที่รับ load สูง

10.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- วางแผน scaling ได้
- ออกแบบบริการที่ scale ได้
- เลือกใช้ auto-scaling

10.4 หัวข้อหลัก

10.4.1 Horizontal vs Vertical Scaling

- **Vertical** — เพิ่ม resource ให้ instance (CPU, RAM)
- **Horizontal** — เพิ่มจำนวน instance
- Microservices เหมาะกับ horizontal

10.4.2 Stateless Services

- บริการไม่เก็บ state ในตัว
- State เก็บใน database/cache
- ทำให้ scale ได้ง่าย
- Session ใช้ Redis หรือ JWT

10.4.3 Auto-Scaling

- **HPA (Horizontal Pod Autoscaler)** ใน Kubernetes
- Scale ตาม CPU, memory, custom metrics
- ต้องมี min/max replicas
- ต้องมี cooldown

10.4.4 Database Scaling

- Read replicas
- Sharding
- Caching (Redis)
- CQRS (แยก read/write)

10.4.5 Scaling Patterns

- **Load Balancing** — กระจาย load
- **Caching** — ลด load ที่ database
- **Rate Limiting** — ป้องกัน overload
- **Circuit Breaker** — หยุดเรียกเมื่อ downstream ล่ม

10.5 ข้อดีของบท

- ทำให้เห็นภาพรวม scaling
- ให้แนวทางปฏิบัติ
- เตรียมพื้นฐานสำหรับ production

10.6 ข้อเสีย/ข้อจำกัด

- ตัวอย่างอาจไม่ครอบคลุมทุกเทคโนโลยี
- ต้องปรับตาม ecosystem

10.7 ข้อควรระวัง

- อย่าเก็บ state ใน service
- อย่าละเลย database scaling
- อย่าละเลย rate limiting
- อย่า auto-scale โดยไม่มี cooldown

10.8 ตัวอย่าง

Order Service:

- Stateless
- ใช้ Redis สำหรับ session
- ใช้ PostgreSQL พร้อม read replica
- Auto-scale ตาม CPU utilization
- Rate limiting ด้วย API Gateway

10.9 การประยุกต์ใช้

- ใช้วางแผน capacity

- ใช้ใน load test
- ใช้ใน architecture review

10.10 สรุป/ผลลัพธ์

ผู้อ่านวางแผน scaling ได้ และออกแบบบริการที่ scale ได้

10.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 11
- ไปสู่เล่ม 1 (Quality attributes)

บทที่ 11: Monitoring และ Observability

11.1 เรื่อง

การ monitoring และ observability สำหรับ microservices

11.2 วัตถุประสงค์

- อธิบายความแตกต่างระหว่าง monitoring กับ observability
- ทำให้เห็นความสำคัญของ observability
- เตรียมพื้นฐานสำหรับ production ที่ดูแลได้

11.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- วางแผน monitoring ได้
- ออกแบบ observability
- ใช้เครื่องมือที่เหมาะสม

11.4 หัวข้อหลัก

11.4.1 Monitoring vs Observability

- **Monitoring** — ตรวจสอบ metrics ที่รู้ล่วงหน้า
- **Observability** — เข้าใจสถานะภายในจาก output ภายนอก
- Observability ครอบคลุมกว่า monitoring

11.4.2 Three Pillars of Observability

- **Metrics** — ตัวเลข เช่น CPU, request rate, latency
- **Logs** — เหตุการณ์ที่เกิดขึ้น

- **Traces** — เส้นทางการเรียกข้ามบริการ

11.4.3 Distributed Tracing

- ติดตาม request ข้ามบริการ
- ใช้ trace ID
- เครื่องมือ: Jaeger, Zipkin, OpenTelemetry
- ช่วย debug ปัญหาใน distributed system

11.4.4 Logging

- Structured logging (JSON)
- Correlation ID
- Centralized logging (ELK, Loki)
- ระดับ log ที่เหมาะสม

11.4.5 Metrics

- RED method: Rate, Errors, Duration
- USE method: Utilization, Saturation, Errors
- เครื่องมือ: Prometheus, Grafana, Datadog

11.4.6 Alerting

- Alert ที่ actionable
- หลีกเลี่ยง alert ที่ noise
- มี runbook
- มี on-call rotation

11.4.7 Health Checks

- Liveness probe — บริการยังทำงานอยู่หรือไม่
- Readiness probe — บริการพร้อมรับ traffic หรือไม่
- Startup probe — บริการเริ่มต้นเสร็จหรือไม่

11.5 ข้อดีของบท

- ทำให้เห็นความสำคัญของ observability
- ให้แนวทางปฏิบัติ
- เตรียมพื้นฐานสำหรับ production

11.6 ข้อเสีย/ข้อจำกัด

- ตัวอย่างอาจไม่ครอบคลุมทุกเทคโนโลยี
- ต้องปรับตาม ecosystem

11.7 ข้อควรระวัง

- อย่าละเลย distributed tracing
- อย่าละเลย correlation ID
- อย่า alert ทุกอย่าง
- อย่าละเลย runbook

11.8 ตัวอย่าง

Observability stack:

- **Metrics:** Prometheus + Grafana
- **Logs:** Loki + Grafana
- **Traces:** Jaeger + OpenTelemetry
- **Alerting:** Alertmanager + PagerDuty

11.9 การประยุกต์ใช้

- ใช้วางแผน observability
- ใช้ใน architecture review
- ใช้ใน production readiness

11.10 สรุป/ผลลัพธ์

ผู้อ่านวางแผน monitoring และ observability ได้

11.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 12–14
- ไปสู่เล่ม 7 (Quality)

ส่วนที่ 4: ความเสี่ยงและการลดความเสี่ยง

บทที่ 12: ข้อดี/ข้อเสียเมื่อเทียบกับ Monolithic

12.1 เรื่อง

การวิเคราะห์ข้อดี/ข้อเสียของ microservices อย่างละเอียด

12.2 วัตถุประสงค์

- ทำให้เห็น trade-offs อย่างครบถ้วน
- ช่วยตัดสินใจได้อย่างมีข้อมูล
- ลดการเลือกตามกระแส

12.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- วิเคราะห์ข้อดี/ข้อเสียได้
- ตัดสินใจได้อย่างมีเหตุผล
- เตรียมความพร้อมสำหรับความท้าทาย

12.4 หัวข้อหลัก

12.4.1 ข้อดีของ Microservices

- **Independent deployability** — deploy แยกได้
- **Technology diversity** — แต่ละบริการเลือก stack ได้
- **Scalability** — scale เฉพาะบริการ
- **Team autonomy** — ทีมดูแลบริการของตนเอง
- **Fault isolation** — บริการหนึ่งล่ม ไม่กระทบทั้งหมด
- **Faster time-to-market** — ทีมเล็ก deploy เร็ว
- **Reusability** — บริการใช้ซ้ำได้

12.4.2 ข้อเสียของ Microservices

- **Complexity** — distributed system ซับซ้อน
- **Operational overhead** — ต้องมี DevOps maturity
- **Data consistency** — eventual consistency
- **Testing** — ต้องมี contract test
- **Debugging** — distributed tracing จำเป็น
- **Network latency** — การเรียกข้าม network
- **Cost** — infrastructure สูง
- **Team coordination** — ต้องมี coordination

12.4.3 ข้อดีของ Monolith

- **Simplicity** — เริ่มต้นง่าย
- **Performance** — in-process calls เร็ว
- **Data consistency** — ACID
- **Debugging** — stack trace ต่อเนื่อง
- **Testing** — integration test ง่าย

- **Cost** — infrastructure ต่ำ
- **Team** — ทีมเดียวจัดการได้

12.4.4 ข้อเสียของ Monolith

- **Coupling** — เมื่อโตขึ้นผูกกันแน่น
- **Deploy** — ทั้งระบบ
- **Scalability** — ทั้งก้อน
- **Technology** — stack เดียว
- **Team scaling** — ทีมใหญ่ทำงานยาก

12.4.5 การตัดสินใจ

การตัดสินใจควรอิง:

- ขนาดระบบ
- ขนาดทีม
- ความพร้อมขององค์กร
- Architectural drivers
- ความเสี่ยงที่ยอมรับได้

12.5 ข้อดีของบท

- ทำให้เห็น trade-offs อย่างครบถ้วน
- ช่วยตัดสินใจได้อย่างมีข้อมูล
- เตรียมความพร้อมสำหรับความท้าทาย

12.6 ข้อเสีย/ข้อจำกัด

- การเปรียบเทียบเป็น contextual
- ตัวอย่างอาจไม่ครอบคลุมทุกกรณี

12.7 ข้อควรระวัง

- อย่าเลือกตามกระแส
- อย่าละเลยความพร้อมขององค์กร
- อย่าละเลยค่าใช้จ่าย

12.8 ตัวอย่าง

กรณีที่ 1: ระบบ e-commerce ขนาดใหญ่ ทีม 50 คน ต้องการ deploy อิสระ → microservices เหมาะสม

กรณีที่ 2: ระบบ ERP ขนาดกลาง ทีม 10 คน ต้องการ simplicity → monolith เหมาะสม

12.9 การประยุกต์ใช้

- ใช้ใน workshop ตัดสินใจ
- ใช้สื่อสารกับผู้มีส่วนได้ส่วนเสีย
- ใช้ใน architecture review

12.10 สรุป/ผลลัพธ์

ผู้อ่านวิเคราะห์ข้อดี/ข้อเสียได้ และตัดสินใจได้อย่างมีเหตุผล

12.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 13–14
- ไปสู่เล่ม 1 (บทที่ 4, 6)

บทที่ 13: แนวทางลดความเสี่ยง

13.1 เรื่อง

แนวทางลดความเสี่ยงในการนำ microservices มาใช้

13.2 วัตถุประสงค์

- ทำให้เห็นความเสี่ยงหลัก
- ให้แนวทางลดความเสี่ยง
- เตรียมความพร้อมสำหรับการนำไปใช้

13.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ระบุความเสี่ยงได้
- วางแนวทางลดความเสี่ยง
- เตรียมความพร้อมสำหรับการนำไปใช้

13.4 หัวข้อหลัก

13.4.1 ความเสี่ยงหลัก

- **Distributed system complexity** — debug ยาก
- **Data consistency** — eventual consistency
- **Operational overhead** — ต้องมี DevOps
- **Network failures** — network ไม่ reliable

- **Team coordination** — ต้องมี coordination
- **Cost overrun** — infrastructure สูง
- **Security** — attack surface กว้างขึ้น

13.4.2 แนวทางลดความเสี่ยง

- **เริ่มด้วย modular monolith** — ค่อย ๆ แยก
- **Strangler Fig Pattern** — ค่อย ๆ แทนที่
- **Invest in DevOps** — automation, observability
- **POC** — พิสูจน์ก่อนลงทุน (ดูเล่ม 6)
- **Training** — ฝึกทีม
- **Start small** — เริ่มจากบริการเล็ก ๆ
- **Measure** — วัดผลก่อน/หลัง

13.4.3 Strangler Fig Pattern

- ค่อย ๆ แทนที่ monolith ด้วย microservices
- ใช้ facade/proxy
- ลดความเสี่ยงของการ rewrite ทั้งหมด
- ทำทีละส่วน

13.4.4 POC

- พิสูจน์แนวคิดก่อนลงทุน
- ใช้เล่ม 6 เป็นแนวทาง
- กำหนด Go/No-Go criteria

13.4.5 Team Topology

- จัดทีมให้สอดคล้องกับบริการ
- Team per service
- Platform team
- Enabling team

13.5 ข้อดีของบท

- ทำให้เห็นความเสี่ยงหลัก
- ให้แนวทางลดความเสี่ยง
- เตรียมความพร้อมสำหรับการนำไปใช้

13.6 ข้อเสีย/ข้อจำกัด

- ตัวอย่างอาจไม่ครอบคลุมทุกองค์กร
- ต้องปรับตามบริบท

13.7 ข้อควรระวัง

- อย่า rewrite ทั้งหมด
- อย่าละเลย DevOps
- อย่าละเลย training
- อย่าละเลยการวัดผล

13.8 ตัวอย่าง

องค์กรที่มี monolith ขนาดใหญ่:

1. เริ่มด้วยการแยกบริการเล็ก ๆ (เช่น Notification)
2. ใช้ Strangler Fig Pattern
3. ลงทุนใน DevOps (CI/CD, observability)
4. ฝึกทีม
5. วัดผล
6. ค่อย ๆ แยกบริการอื่น

13.9 การประยุกต์ใช้

- ใช้วางแผน migration
- ใช้ใน POC
- ใช้ใน architecture review

13.10 สรุป/ผลลัพธ์

ผู้อ่านระบุความเสี่ยงได้ และวางแนวทางลดความเสี่ยง

13.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 14
- ไปสู่เล่ม 5 (ความเสี่ยง)
- ไปสู่เล่ม 6 (POC)

บทที่ 14: Distributed System Challenges

14.1 เรื่อง

ความท้าทายของ distributed system ที่ microservices ต้องเผชิญ

14.2 วัตถุประสงค์

- ทำให้เห็นความท้าทายของ distributed system
- ให้แนวทางจัดการ
- เตรียมความพร้อมสำหรับ production

14.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ระบุความท้าทายได้
- วางแนวทางจัดการ
- เตรียมความพร้อมสำหรับ production

14.4 หัวข้อหลัก

14.4.1 Fallacies of Distributed Computing

1. Network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. Network is secure
5. Topology doesn't change
6. There is one administrator
7. Transport cost is zero
8. Network is homogeneous

ทุกข้อเป็นความเชื่อที่ผิด

14.4.2 CAP Theorem

- Consistency, Availability, Partition tolerance
- เลือกได้ 2 จาก 3
- ใน distributed system ต้องยอมรับ trade-off

14.4.3 PACELC Theorem

- ขยาย CAP
- ถ้า Partition: เลือก A หรือ C
- ถ้าไม่ Partition: เลือก Latency หรือ Consistency

14.4.4 Distributed Transactions

- ไม่มี ACID ข้ามบริการ
- ใช้ Saga pattern
- ใช้ eventual consistency

- ต้องออกแบบ compensating actions

14.4.5 Network Failures

- Timeout, retry, circuit breaker
- Idempotency
- Fallback

14.4.6 Clock Synchronization

- ไม่มี global clock
- ใช้ logical clock (Lamport, Vector)
- ระวัง timestamp

14.4.7 Partial Failures

- บางส่วนล่ม บางส่วนทำงาน
- ต้องออกแบบให้ tolerate
- ใช้ bulkhead, circuit breaker

14.5 ข้อดีของบท

- ทำให้เห็นความท้าทายของ distributed system
- ให้แนวทางจัดการ
- เตรียมความพร้อมสำหรับ production

14.6 ข้อเสีย/ข้อจำกัด

- ตัวอย่างอาจไม่ครอบคลุมทุกกรณี
- ต้องปรับตามบริบท

14.7 ข้อควรระวัง

- อย่าละเลย fallacies
- อย่าละเลย CAP theorem
- อย่าละเลย partial failures
- อย่าละเลย clock synchronization

14.8 ตัวอย่าง

Order Service เรียก Payment Service:

- **Network failure:** ใช้ timeout + retry + circuit breaker
- **Partial failure:** ใช้ bulkhead + fallback
- **Clock:** ใช้ logical clock สำหรับ event ordering

14.9 การประยุกต์ใช้

- ใช้ออกแบบ resilient services
- ใช้ใน architecture review
- ใช้ใน POC

14.10 สรุป/ผลลัพธ์

ผู้อ่านระบุความท้าทายได้ และวางแนวทางจัดการ

14.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 15–16
- ไปสู่เล่ม 5 (ความเสี่ยง)

ส่วนที่ 5: การนำไปใช้และกรณีศึกษา

บทที่ 15: ตัวอย่างการออกแบบระบบ Microservices

15.1 เรื่อง

กรณีศึกษาการออกแบบระบบ microservices ตั้งแต่ต้นจนจบ

15.2 วัตถุประสงค์

- ประยุกต์แนวคิดทั้งหมดกับกรณีศึกษา
- ทำให้เห็นภาพรวมการออกแบบ
- เตรียมผู้อ่านสำหรับการออกแบบระบบของตนเอง

15.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ออกแบบระบบ microservices ได้
- ใช้เครื่องมือและแนวทางจากบทก่อน ๆ
- วิเคราะห์ trade-offs

15.4 หัวข้อหลัก

15.4.1 บริบท

ระบบ e-commerce สำหรับ SME:

- ผู้ใช้: 1,000 คน/วัน
- ทีม: 15 คน แบ่งเป็น 3 ทีม
- ต้องการ deploy อิสระ
- ต้องการ scale เฉพาะส่วน

15.4.2 การกำหนดขอบเขตบริการ

- **Catalog Service** — จัดการสินค้า
- **Order Service** — จัดการคำสั่งซื้อ
- **Payment Service** — จัดการชำระเงิน
- **Notification Service** — แจ้งเตือน
- **User Service** — จัดการผู้ใช้

15.4.3 API Design

ตัวอย่าง API ของ Order Service:

```
POST /v1/orders
GET /v1/orders/{id}
PUT /v1/orders/{id}/status
```

15.4.4 Data Ownership

- แต่ละบริการมีฐานข้อมูลของตนเอง
- Order Service ไม่ query ฐานข้อมูล Payment โดยตรง
- ใช้ event สำหรับการสื่อสาร

15.4.5 Communication

- Order → Payment: synchronous (ต้องการผลทันที)
- Order → Notification: asynchronous (ไม่ต้องการผลทันที)
- Payment → Order: event (`PaymentProcessed`)

15.4.6 Deployment

- Docker + Kubernetes
- CI/CD pipeline
- Blue/Green deployment

15.4.7 Scaling

- Order Service: auto-scale ตาม CPU
- Catalog Service: read replica + cache
- Payment Service: scale ตาม queue length

15.4.8 Monitoring

- Prometheus + Grafana
- Jaeger สำหรับ tracing
- Loki สำหรับ logs

15.4.9 ความเสี่ยงและมาตรการ

- **Network failure:** timeout + retry + circuit breaker
- **Data consistency:** Saga pattern
- **Operational overhead:** ลงทุนใน DevOps

15.5 ข้อดีของบท

- ทำให้เห็นภาพรวมการออกแบบ
- เชื่อมแนวคิดทั้งหมดเข้าด้วยกัน
- เตรียมผู้อ่านสำหรับการออกแบบระบบของตนเอง

15.6 ข้อเสีย/ข้อจำกัด

- เป็นกรณีศึกษาเดียว
- ตัวอย่างอาจไม่ครอบคลุมทุกกรณี

15.7 ข้อควรระวัง

- อย่าคัดลอกโดยไม่ปรับบริบท
- อย่าละเลย trade-offs
- อย่าละเลยการวัดผล

15.8 ตัวอย่าง

Order flow:

1. User สร้าง order ผ่าน Order Service
2. Order Service เรียก Payment Service (synchronous)
3. Payment Service ประมวลผลและส่ง event `PaymentProcessed`
4. Order Service รับ event และยืนยัน order
5. Order Service ส่ง event `OrderConfirmed` ให้ Notification Service
6. Notification Service ส่งอีเมล

15.9 การประยุกต์ใช้

- ใช้เป็น template สำหรับระบบของตนเอง
- ใช้ใน workshop
- ใช้ใน architecture review

15.10 สรุป/ผลลัพธ์

ผู้อ่านออกแบบระบบ microservices ได้ และเห็นภาพรวมการทำงาน

15.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 16
- ไปสู่เล่ม 3–4 (DDD, Clean Architecture)

บทที่ 16: การประยุกต์ใช้และสรุปเล่ม

16.1 เรื่อง

การนำเนื้อหาทั้งเล่มไปใช้ในองค์กร

16.2 วัตถุประสงค์

- สรุปสาระสำคัญของเล่ม
- แปลงความรู้เป็นแผนปฏิบัติ
- เชื่อมไปยังเล่มถัดไป

16.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- มีแผนประยุกต์ใช้ในองค์กร
- จัด workshop ตามแม่แบบในเล่ม
- รู้ว่าต้องไปอ่านเล่มใดต่อ

16.4 หัวข้อหลัก

16.4.1 Workshop: Service Boundary Canvas

- ใช้ภาคผนวก A
- ระบุ business capability
- กำหนดขอบเขตบริการ
- ระบุ data ownership

16.4.2 Workshop: API Contract Review

- ใช้ภาคผนวก B
- ตรวจ API contract
- กำหนด versioning
- กำหนด security

16.4.3 Workshop: Communication Pattern Decision

- ใช้ภาคผนวก D
- เปรียบเทียบ synchronous vs asynchronous
- บันทึกการตัดสินใจ

16.4.4 Workshop: Deployment Readiness

- ใช้ภาคผนวก E
- ตรวจความพร้อมของ deployment
- ระบุ gaps

16.4.5 สรุปสาระสำคัญ

- Microservices เป็นเครื่องมือ ไม่ใช่เป้าหมาย
- การตัดสินใจต้องอิง drivers และ trade-offs
- Data ownership เป็นหัวใจ
- Communication pattern สำคัญ
- Observability จำเป็น
- ความเสี่ยงต้องจัดการ

16.4.6 เชื่อมไปเล่ม 3–4

- เล่ม 3: DDD สำหรับการกำหนดขอบเขต
- เล่ม 4: Clean Architecture สำหรับโครงสร้างภายใน
- เล่ม 5: ความเสี่ยง
- เล่ม 6: POC
- เล่ม 7: การบริหารโครงการ

16.5 ข้อดีของบท

- ปิดเล่มด้วยการลงมือทำ
- ทำให้ความรู้แปลงเป็นแผนปฏิบัติ
- เชื่อมโยงกับเล่มถัดไปอย่างชัดเจน

16.6 ข้อเสีย/ข้อจำกัด

- ต้องปรับตามบริบทองค์กร

- ต้องมีผู้นำ workshop

16.7 ข้อควรระวัง

- อย่านำไปใช้แบบคัดลอกโดยไม่ปรับบริบท
- อย่าทำ workshop ครั้งเดียวแล้วจบ
- อย่าละเลยการติดตามผล

16.8 ตัวอย่าง

แผนนำไปใช้ใน 90 วัน:

- **เดือน 1:** Service Boundary Canvas + Communication Pattern Decision
- **เดือน 2:** POC สำหรับบริการเล็ก ๆ
- **เดือน 3:** เริ่มแยกบริการแรก + ตั้ง observability

16.9 การประยุกต์ใช้

- ใช้เป็นแผนดำเนินงานหลังอ่านจบ
- ใช้เป็น checklist ของทีม
- ใช้เป็นพื้นฐานสำหรับเล่มถัดไป

16.10 สรุป/ผลลัพธ์

ผู้อ่านมีแผนประยุกต์ใช้ในองค์กร และรู้เส้นทางการอ่านต่อ

16.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่เล่ม 3–7 ทั้งหมด

ภาคผนวก

ภาคผนวก A: Service Boundary Canvas

หัวข้อ	คำอธิบาย	ตัวอย่าง
ชื่อบริการ	ชื่อที่สื่อ business capability	Order Service
Business Capability	ความสามารถทางธุรกิจที่รับผิดชอบ	จัดการคำสั่งซื้อ
Data Ownership	ข้อมูลที่บริการเป็นเจ้าของ	Orders, OrderItems

หัวข้อ	คำอธิบาย	ตัวอย่าง
API	interface ที่เปิดให้บริการ	REST API
Dependencies	บริการที่ต้องพึ่งพา	Payment, Inventory
Consumers	บริการที่เรียกใช้	Notification, Reporting
Team	ทีมที่รับผิดชอบ	Order Team
Change Frequency	ความถี่ในการเปลี่ยนแปลง	สูง

ภาคผนวก B: API Contract Checklist

- [ ] ใช้ resource-oriented design
- [ ] มี versioning strategy
- [ ] มี documentation (OpenAPI)
- [ ] มี authentication/authorization
- [ ] มี rate limiting
- [ ] มี input validation
- [ ] มี error response ที่ชัดเจน
- [ ] มี examples
- [ ] ใช้ HTTPS
- [ ] มี idempotency สำหรับ operation ที่เหมาะสม
- [ ] มี backward compatibility plan

ภาคผนวก C: Data Ownership Matrix

ข้อมูล	เจ้าของ	ผู้ใช้	วิธีการเข้าถึง
Orders	Order Service	Payment, Notification	API/Event
Payments	Payment Service	Order, Reporting	API/Event
Products	Catalog Service	Order, Search	API
Users	User Service	ทุกบริการ	API
Inventory	Inventory Service	Order	API/Event

ภาคผนวก D: Communication Pattern Decision Record

หัวข้อ	รายละเอียด
ชื่อ decision	การสื่อสารระหว่าง Order และ Payment
ปัญหา	ต้องเลือกระหว่าง synchronous และ asynchronous
ทางเลือก	1. REST synchronous 2. Event asynchronous
เหตุผล	ต้องการผลทันทีสำหรับการยืนยัน order
ข้อดี/ข้อเสีย	Synchronous: ง่าย แต่ coupling สูง
ความเสี่ยง	Payment ล่ม → Order ล่ม
การตัดสินใจ	Synchronous + circuit breaker + fallback
ผู้รับผิดชอบ	Architect + Tech Lead
วันที่	2026-10-30

ภาคผนวก E: Deployment Readiness Checklist

- [ ] มี container image
- [ ] มี health check (liveness, readiness)
- [ ] มี graceful shutdown
- [ ] มี resource limits
- [ ] มี logging
- [ ] มี monitoring
- [ ] มี tracing
- [ ] มี secrets management
- [ ] มี rollback plan
- [ ] มี CI/CD pipeline
- [ ] มี documentation

ภาคผนวก F: Observability Checklist

- [ ] มี metrics (RED/USE)
- [ ] มี logs (structured, correlation ID)
- [ ] มี traces (distributed tracing)
- [ ] มี dashboards
- [ ] มี alerting
- [ ] มี runbook
- [ ] มี on-call rotation
- [ ] มี SLO/SLI
- [ ] มี error budget

ภาคผนวก G: Glossary เฉพาะเล่ม 2

คำ	ความหมายโดยย่อ
Microservice	บริการขนาดเล็กที่ทำงานอิสระ
Service Boundary	ขอบเขตของบริการ
API	Contract ระหว่างบริการ
Data Ownership	การเป็นเจ้าของข้อมูลของบริการ
Eventual Consistency	ข้อมูลสอดคล้องกันในที่สุด
Saga	ลำดับ local transactions ที่ประสานกัน
Compensating Transaction	การชดเชยเมื่อ saga ล้มเหลว
Circuit Breaker	หยุดเรียกเมื่อ downstream ล่ม
Bulkhead	แยก resource pool
Idempotency	การเรียกซ้ำได้ผลเหมือนเดิม
Distributed Tracing	การติดตาม request ข้ามบริการ
Observability	ความสามารถในการเข้าใจสถานะภายใน
SLO	Service Level Objective

คำ	ความหมายโดยย่อ
SLI	Service Level Indicator
Error Budget	งบประมาณข้อผิดพลาด
Strangler Fig Pattern	การค่อย ๆ แทนที่ระบบเดิม
Conway's Law	องค์กรออกแบบระบบสะท้อนโครงสร้างการสื่อสาร
CAP Theorem	Consistency, Availability, Partition tolerance เลือกได้ 2
PACELC Theorem	ขยาย CAP ด้วย Latency/Consistency
Fallacies of Distributed Computing	ความเชื่อผิด ๆ เกี่ยวกับ distributed system

---

เกณฑ์ประเมินความสำเร็จของเล่ม

เกณฑ์	ตัวชี้วัด	น้ำหนัก
ความเข้าใจ	อธิบายหลักการ microservices และ trade-offs ได้	20%
การออกแบบ	มี Service Boundary Canvas และ API contract	25%
Data Ownership	มี Data Ownership Matrix	15%
การสื่อสาร	มี Communication Pattern Decision Record	15%
Operations	มี Deployment Readiness และ Observability Checklist	15%
ความเสี่ยง	มีแนวทางลดความเสี่ยง	10%

สรุปเล่ม 2

เล่ม 2 ทำหน้าที่เป็นคู่มือสำหรับการออกแบบ พัฒนา และดำเนินงาน microservices อย่างมีเหตุผล ผู้อ่านจะได้:

1. **ความเข้าใจหลักการ** — microservices คืออะไร ไม่ใช่อะไร
2. **การกำหนดขอบเขต** — ใช้ DDD และ business capability
3. **การออกแบบ API** — contract, versioning, security

4. **Data Ownership** — การแยกข้อมูลและการจัดการ consistency
5. **การสื่อสาร** — synchronous, asynchronous, event-driven, Saga
6. **Operations** — deployment, scaling, monitoring
7. **การจัดการความเสี่ยง** — trade-offs, แนวทางลดความเสี่ยง, distributed system challenges
8. **กรณีศึกษา** — ตัวอย่างการออกแบบระบบจริง

เมื่ออ่านจบ ผู้อ่านควรมีความพร้อมในการตัดสินใจและดำเนินงาน microservices อย่างมีเหตุผล และรู้ว่าเมื่อใดควรเลือก monolith แทน
